

M13 - Delta Lake + Lakehouse

Zero-to-Job-Ready Data Engineer 2026 | RC2 | 11 beginner-complete lessons | 30-40 focused hours

BEGINNER SUFFICIENCY RULE

This book is the primary teaching source. RC1 was rejected because the explanation and labs were too compressed. Read the concept before opening the code. Videos demonstrate; labs build skill.

PINNED RUNTIME

M13 intentionally uses a separate environment from M12: `pyspark==4.0.0` and `delta-spark==4.0.0`. The current Delta 4.0.x compatibility line targets Spark 4.0.x. Do not force the M12 PySpark 4.2 environment to run M13.

M13-L01 - Data Lake vs Warehouse vs Lakehouse

Why this matters

You need to know what problem Delta is solving before learning its API.

Start with the simple idea

A data lake is usually flexible file/object storage. A warehouse adds managed table behavior and SQL-oriented governance/performance. A lakehouse keeps lake-style storage but adds table reliability so engineering and analytics can share governed data. None of these labels is automatically the best answer.

Technical model

A lakehouse is an architecture pattern, not simply a folder full of Parquet files.

Delta Lake stores table data as files plus transaction-log metadata. The log lets readers reason about committed table versions rather than guessing from loose files.

Warehouses and lakehouses can coexist. One system may preserve raw events in object storage, curate Delta tables with Spark, and expose downstream SQL/BI through a warehouse or SQL endpoint.

Code pattern

```
bronze_path = 'lake/bronze/events'
silver_path = 'lake/silver/events'
```

Worked reasoning example

A retailer receives JSON clickstream, CSV product files and BI requests. Raw source copies can stay in lake storage for replay. Spark can standardize trusted Delta tables. Finance may still prefer a governed SQL-facing warehouse. The design is driven by workload and consumers, not by the word 'lakehouse'.

Common mistakes and debugging

- Calling any Parquet directory a lakehouse.
- Assuming lakehouse means streaming is mandatory.
- Replacing every warehouse because a newer architecture exists.

Engineering decision

Choose the platform that satisfies consumers, operational reliability, cost and team skills. Keep the logical layer contracts portable.

Before practice

Draw Source -> Raw -> Bronze Delta -> Silver Delta -> Gold Delta -> Consumers. State the grain and owner of each stage.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.

M13-L02 - Why Plain Files Are Not Tables

Why this matters

Distributed files can store data efficiently while still lacking reliable table transactions and history.

Start with the simple idea

Imagine updating a dataset stored across 20 Parquet files. If your job overwrites 7 files and crashes before the remaining 13 are safely replaced, a reader can see an inconsistent mixture. A table layer needs a committed view of the whole logical dataset.

Technical model

Plain files do not automatically provide one atomic commit across multi-file changes.

A transaction log records committed metadata/actions so readers use a consistent table version.

Table metadata can also carry schema and operation history that loose-file naming conventions cannot safely replace.

Code pattern

```
df.write.mode('overwrite').parquet('plain_path')  
# compare with a Delta table write in G01
```

Worked reasoning example

A daily customer table is rewritten while a BI query starts. With unmanaged files the reader may discover partially replaced data depending on the write pattern. With Delta, the new state becomes visible as a committed table version rather than an arbitrary halfway folder state.

Common mistakes and debugging

- Editing files inside a Delta table directory manually.
- Assuming a successful single-file test proves a multi-file transaction is safe.
- Treating `\_delta\_log` as disposable clutter.

Engineering decision

Once a directory is a managed Delta table, mutate it through supported Delta/Spark operations rather than manual file surgery.

Before practice

In G01, identify both the data files and `\_delta\_log`. Explain why both belong to one logical table.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.

M13-L03 - Transaction Log, ACID and Table Versions

Why this matters

The log is the foundation for consistent reads, mutation history and recovery reasoning.

Start with the simple idea

Think of each successful change as creating a new committed table version. Readers ask what the table looked like at a committed version, not which files happen to exist at this instant.

Technical model

Atomicity means a transaction is committed as a whole or not exposed as a successful table state.

Consistency here includes enforcing table/schema rules and valid transaction transitions.

Isolation allows readers to work from a consistent snapshot while other writes occur.

Durability means committed state is represented persistently in storage/log metadata.

Code pattern

```
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, path)
dt.history().select('version','operation').show()
```

Worked reasoning example

Version 0 creates a table. Version 1 updates a product. Version 2 merges a daily batch. A failed uncommitted attempt should not be treated as version 3. Your operational evidence is the Delta history, not only a console message that a script reached the end.

Common mistakes and debugging

- Reading random data files instead of the Delta table path.
- Assuming every filesystem change equals a Delta version.
- Deleting log files to save space without understanding retention and recovery consequences.

Engineering decision

Use Delta history and table state as first-class operational evidence for debugging and audit.

Before practice

In G02, predict version numbers before running. Save version + operation output and compare with your prediction.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.

M13-L04 - Create, Read, Update and Delete

Why this matters

Reliable lakehouse tables need controlled mutation, not blind append-only habits or manual rewrites.

Start with the simple idea

Create/write defines the table. Read loads the current committed snapshot. Update changes matching records. Delete removes matching logical rows. Every mutation should be limited by an explicit business condition and validated before/after.

Technical model

``DataFrameWriter.format("delta")`` creates/appends table data.

``DeltaTable.forPath`` gives access to update/delete/merge/history APIs.

Update/delete conditions are business logic. Validate the intended row population before executing broad mutation.

Code pattern

```
dt = DeltaTable.forPath(spark, path)
dt.update(condition='status = \'OLD\'', set={'status': "'\ARCHIVED'"})
```

Worked reasoning example

If a status correction targets ``order_id='O7'``, first prove O7 is one intended row. A condition such as ``status IS NOT NULL`` may modify an entire table even though the syntax is valid. Correctness requires controlled scope plus after-checks.

Common mistakes and debugging

- Running update/delete without measuring target rows.
- Using broad conditions because the lab dataset is tiny.
- Assuming deleted data should be recovered by manually copying old Parquet files.

Engineering decision

Before destructive mutation, establish target grain/count and save history evidence after the commit.

Before practice

In G03, record current count, intended mutation count, final rows and history. Explain what would make you rollback/rebuild instead.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.

M13-L05 - Schema Enforcement and Schema Evolution

Why this matters

A data platform must reject incompatible surprises while still supporting approved change.

Start with the simple idea

Schema enforcement says incoming data must satisfy the table contract. Schema evolution says the contract can change intentionally. Those are different ideas: evolution is not permission for any new or incompatible field to flow silently.

Technical model

Delta validates write schema against the target table.

Operation-scoped ``mergeSchema`` can intentionally add compatible columns for a specific write.

Type changes, renamed business meaning and unexpected fields still require source-contract review and downstream impact analysis.

Code pattern

```
new.write.format('delta').option('mergeSchema',true).mode('append').save(path)
```

Worked reasoning example

Yesterday a customer feed has ``id,name``; today it adds ``currency``. If the field is approved and nullable for old rows, a controlled add-column evolution may be safe. If ``amount`` changes from decimal to free text, enabling evolution is not a substitute for validation and remediation.

Common mistakes and debugging

- Turning on automatic schema evolution globally without governance.
- Forgetting old rows may have null for newly added fields.
- Confusing column-name addition with safe semantic compatibility.

Engineering decision

Prefer explicit contract changes. Record what changed, why it is compatible, how old rows behave and what downstream consumers must adjust.

Before practice

In G05, add one approved field, inspect schema before/after, then deliberately try one incompatible case in a disposable copy.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.

M13-L06 - Time Travel and Version History

Why this matters

Historical snapshots are valuable for debugging, audit and controlled recovery.

Start with the simple idea

Time travel lets you query an older committed table version. That means you can compare before/after states without guessing from saved screenshots. It is powerful investigation evidence, but it is not a permanent backup promise.

Technical model

Read older state using ``versionAsOf`` or supported timestamp-based history within the available retention window.

History tells you operations/versions; time travel reads a specific older snapshot.

Vacuum/retention policies can eventually remove older files required to reconstruct past versions.

Code pattern

```
old = spark.read.format('delta').option('versionAsOf',0).load(path)
```

Worked reasoning example

A bad merge in version 5 changes 300 rows. You can compare version 4 to version 5 to isolate the change. Recovery may restore/reprocess from a known version, but long-term disaster recovery still needs separate backup/replication strategy.

Common mistakes and debugging

- Calling time travel permanent backup.
- Hard-coding version numbers without first checking history.
- Deleting old files manually and then expecting historical reads to work.

Engineering decision

Use time travel for investigation and reproducible comparisons; design backup/retention separately.

Before practice

In G04, compare current and version 0 counts/values, then write one sentence explaining what evidence could disappear after retention cleanup.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.

M13-L07 - MERGE, Upserts and Idempotent Incremental Loads

Why this matters

Daily changes often contain both updates to existing keys and new keys. MERGE is the core pattern for applying them deterministically.

Start with the simple idea

A MERGE matches source records to target records using a business/key condition. Matching rows can update; unmatched rows can insert. The hard part is not the API call - it is ensuring the source has one deterministic intended row per target key.

Technical model

The merge condition defines identity, e.g. ``t.product_id = s.product_id``.

``whenMatchedUpdateAll`` updates matching rows; ``whenNotMatchedInsertAll`` inserts new ones.

Duplicate source rows for the same key can make the intended update ambiguous. Deduplicate or resolve source precedence before MERGE.

Rerun proof: replaying the exact same source batch should not create additional target keys or corrupt state.

Code pattern

```
(dt.alias('t').merge(src.alias('s'), 't.id=s.id')
  .whenMatchedUpdateAll()
  .whenNotMatchedInsertAll()
  .execute())
```

Worked reasoning example

Target starts with P1,P2,P3. Source contains P2 update and P4 insert twice. Before MERGE, resolve P4 to one deterministic source record. First run ends with four products. Replaying the same batch should remain four products with the same current values.

Common mistakes and debugging

- Appending an update batch instead of merging it.
- Using a non-unique source without a precedence rule.
- Proving idempotency only by “the command did not crash.”

Engineering decision

Define identity, source dedupe/precedence and rerun checks before writing MERGE code.

Before practice

In G06, run the same merge twice and record target row count + distinct-key count after each run.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.

M13-L08 - Slowly Changing Dimension Type 2 with Delta

Why this matters

Historical dimension changes need a deliberate history model, not a normal current-state upsert.

Start with the simple idea

Type 2 keeps old versions and creates a new version when tracked attributes change. One business key can therefore have multiple historical rows but only one current row. Facts can keep pointing to the historical version that was valid at event time.

Technical model

Typical columns: business key, surrogate/history identifier where used, tracked attributes, ``effective_from``, ``effective_to``, ``is_current``.

For a changed current row, expire the old version and insert the new version. New business keys insert directly.

A MERGE alone is not automatically SCD2; the operation must intentionally create history versions and preserve one-current-row uniqueness.

Code pattern

```
# concept
# 1 expire changed current rows
# 2 insert new current versions
# 3 validate one is_current row per business key
```

Worked reasoning example

Customer C1 changes city from Deoghar to Ranchi on Aug 27. Old row becomes non-current with an end date. New Ranchi row starts Aug 27 and is current. Sales from January can still join to the older customer version if modeled correctly.

Common mistakes and debugging

- Overwriting attributes with Type 1 when historical reporting requires old values.
- Leaving two current rows for one customer.
- Expiring a row even when tracked attributes did not change.

Engineering decision

Use SCD2 only for attributes whose historical meaning matters. Enforce one current row per business key and test boundary dates.

Before practice

In G07, inspect current state, design expire+insert steps, then complete the logic in a copy and validate current-row uniqueness.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.

M13-L09 - Bronze, Silver and Gold Layer Contracts

Why this matters

Layer names are useful only when each layer has an explicit contract.

Start with the simple idea

Bronze preserves source fidelity and replayability. Silver produces trusted standardized entities/events. Gold exposes consumer-ready business grain and metrics. A color name alone does not define quality.

Technical model

Bronze may keep duplicates/malformed source records intentionally if fidelity is required, while recording ingestion metadata.

Silver applies schema, dedupe, normalization, referential/business validation and chosen current/history semantics.

Gold represents specific consumer grains such as daily category revenue or current inventory summary.

Code pattern

```
raw.write.format('delta').save(bronze)
silver.write.format('delta').save(silver_path)
gold.write.format('delta').save(gold_path)
```

Worked reasoning example

A malformed `customer\_id` may remain visible in Bronze for replay/audit but be quarantined before Silver. Gold should not re-clean raw strings; it should consume trusted Silver entities and calculate a defined business output.

Common mistakes and debugging

- Cleaning Bronze until the original source can no longer be reproduced.
- Letting Silver contain unvalidated duplicates because “Gold will fix it.”
- Building Gold without stating its grain/consumer.

Engineering decision

Define allowed transformations, rejected-record behavior, uniqueness, freshness and grain at every layer boundary.

Before practice

In G08, introduce one bad raw row and state whether it stays in Bronze, enters Silver, and can contribute to Gold.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.

M13-L10 - File Layout, Compaction and Table Maintenance

Why this matters

Transactional correctness does not guarantee efficient physical layout.

Start with the simple idea

A Delta table still consists of data files. If streaming/microbatch jobs create thousands of tiny files, readers pay metadata/open-file overhead. If you force one enormous file, parallelism suffers. Maintenance is evidence-driven.

Technical model

Partition columns affect physical directory pruning and should match common filters without creating extreme high-cardinality folders.

Compaction combines many small files into fewer reasonably sized files; exact commands/features can vary by platform.

VACUUM-style cleanup and retention interact with time-travel availability. Treat retention as an operational policy, not a housekeeping afterthought.

Inspect actual file counts, sizes, query plans and workload filters before changing layout.

Code pattern

```
df.repartition(4).write.format('delta').mode('overwrite').save(path)
```

Worked reasoning example

A 1-minute ingestion job creates 40,000 tiny files/day. First measure file counts/sizes and read behavior. You may compact or adjust upstream write batching/partitioning. `coalesce(1)` makes a demo tidy but is not a scalable maintenance strategy.

Common mistakes and debugging

- Partitioning by unique transaction ID.
- Using `coalesce(1)` as the universal small-file fix.
- Vacuuming aggressively without understanding time-travel/recovery requirements.

Engineering decision

Separate logical correctness from physical maintenance. Optimize only after measuring the workload and preserving recovery requirements.

Before practice

In G09, compare requested partition count with actual data/log files and explain why tiny test-data performance cannot be generalized.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.

M13-L11 - Lakehouse Integration and Debugging

Why this matters

Job-ready skill means connecting transactions, schema, merge, history, quality and layers into one repeatable pipeline.

Start with the simple idea

A production-style Delta pipeline starts from source evidence, lands raw data, validates it, transforms trusted Silver state/history, builds Gold outputs, records history/audit evidence, and can be rerun or backfilled without duplicating logical entities.

Technical model

Debug from evidence: source counts/schema -> Bronze count/history -> Silver quality/key uniqueness -> Gold grain/control totals -> table history/file layout.

A failed downstream Gold step should not force you to guess whether Bronze/Silver committed. Inspect each table state/history.

Runtime compatibility is part of engineering. This module uses Spark 4.0 + Delta 4.0 in its own environment rather than ignoring package compatibility.

Code pattern

```
# source -> bronze -> validate -> silver MERGE -> gold -> history/audit  
# run same batch again and compare key/count controls
```

Worked reasoning example

An inventory update has three input rows but two unique product IDs because P4 is duplicated. Bronze preserves all three. Silver resolves to one P4 and applies P2/P4 changes. Gold has category metrics. Replaying the input leaves four Silver products. If counts change on replay, the pipeline failed idempotency.

Common mistakes and debugging

- Debugging only the last stack trace without checking table history.
- Ignoring source duplicates because MERGE syntax looks correct.
- Mixing incompatible Spark/Delta versions because both packages install separately.

Engineering decision

A reliable lakehouse pipeline has explicit source, layer, schema, key, rerun, history and maintenance contracts.

Before practice

Before the Module Practical, write your expected source rows, unique keys, final Silver keys, Gold grain and version progression before running code.

Explain it without notes

Explain the concept in 3-5 sentences, name the Delta evidence you would inspect, and describe one unsafe choice that could corrupt history, duplicate current-state rows, or make replay unreliable.